

reboot

REFERENCE



API consumer



perform a warm reset of the reader

Overview

`reboot` performs a **warm reset** of the reader. It requires only `command` and `requestId`. The device **accepts** the request (`response.code 1`), reboots, and then **auto-reconnects to the previously-connected server** — no manual reconnect needed. It is a state-changing command on the device-management ([MGMT](#)) endpoint.

`reboot` is **session-severing** — the reply is `code 1` ("Command payload is accepted"), returned before the reset, and the control session drops for ~30 s–2 min. See the warning in [Notes and Caveats](#).

Details

Property	Value
Interaction Pattern	Asynchronous Request → Accept → Warm reset → Auto-reconnect
State Change	STATE-CHANGING (warm reset; clears volatile state)
Required Request Fields	<code>command</code> , <code>requestId</code> (no other payload)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>set os</code> , <code>get version</code> , <code>get status</code> , <code>control operation</code>

When to Use

Use `reboot` to **warm-reset the reader**:

- **Recover from a bad state** — clear a transient fault when a targeted command will not.
- **Apply/settle changes** that take effect on restart.

- Clear the [operating mode](#) — a reboot resets the operating mode to default (see the volatility note below).

Do **not** reboot during an active RFID inventory — the device refuses with [code 5](#). Stop the inventory first with [control operation](#).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	The code 1 accept, returned before the reset. Correlated by the in-payload requestId .

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

`reboot` carries **no credential field** — there is nothing to mask. The `requestId` you send is echoed in the accept so you can pair reply to request.

See [Topics and Channels](#).

Key Fields

- `command` / `requestId` (request) — the entire request; there is no reboot payload.
- `response.code` (response) — `1` means *accepted* (the reset then proceeds); `5` means the device refused the reboot because an inventory is running. See [Response Codes](#).

Request Payload

The request is a minimal envelope — just `command` and `requestId`.

Request payload

```
1 {
2   "command" : "reboot" ,
3   "requestId" : "123abcd"
4 }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	reboot	Fixed command name.
2	requestId *	string	—	123abcd	A unique identifier for the request; echoed back in the response. There is no other payload.

Response Payload

The reply is an **accept** — `command`, `requestId`, `apiVersion`, and `response` with code 1, returned *before* the reset begins.

Response payload

Example: accept

```
1 {
2   "command" : "reboot" ,
3   "requestId" : "123abcd" ,
4   "apiVersion" : "V1.1" ,
5   "response" : {
6     "code" : 1 ,
7     "description" : "Command payload is accepted"
8   }
9 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	reboot	Echoes the executed command.
2	requestId *	string	—	123abcd	Echoes the request's requestId — pairs reply to request.
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1	API version reported by this command. Emitted per-handler, not device-wide.
4	response *	object	—	—	Accept/status envelope. This is an accept returned before the reset — not a "reboot complete" result (see below).
5	code *	integer	0–101	0	1 = the payload was accepted (the reboot success-ack — not 0). See Response Codes below.
6	description *	string	—	Success	Human-readable status.

Lifecycle

A single `reboot` drives the device through a clean warm-reset cycle.

Acknowledgment

The device replies `response.code 1` ("Command payload is accepted"), *before* it resets. This is the success-ack — there is no later "reboot complete" message on success. If the reboot fails, the device sends a failure notification.

Session Impact

- **Down** — the reader drops its MQTT connections and warm-resets. It is unreachable for the duration.
- **Auto-reconnect** — the device auto-reconnects to the **previously-connected server on its own**, with no manual intervention. Recovery time **varies** (observed ~44 s and ~110 s across two runs — roughly ~30 s to ~2 min); do not hard-code a fixed wait.

Persistence

State after reset:

- **Endpoint configuration persists** — saved and active endpoints are restored/re-activated across the reset.
- **Operating Mode is volatile** — the reboot resets the operating mode to default (e.g. `BALANCED_PERFORMANCE`, any armed `accessOperations` cleared). See `set operating mode`; this is the documented "on reboot the set configuration is lost" behavior.

Response Codes

`reboot` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
1	Command payload is accepted (<code>IOT_STATUS_CMD_PAYLOAD_ACCEPTED</code>) — the reboot success-ack , returned before the reset. The device then reboots and auto-reconnects.
0	Success — the generic success code; in practice <code>reboot</code> acks with 1, not 0.

Code	Description
5	Can't reboot device, inventory in progress (<code>IOT_ERROR_IN_REBOOT_INVENTORY_IN_PROGRESS</code>) — unique to <code>reboot</code> ; the device refuses the reset while an RFID inventory is running. Stop the inventory with <code>control_operation</code> , then retry.
101	Default error — generic catch-all.

[Code 1](#) is the acceptance ack, not a `reboot` -only code. The contract scopes code 1 ("Command payload is accepted") to `reboot` , `set_os` and `install_certificate` alike, and scopes [code 5](#) to `reboot` alone. Accept 0 and 1 as success-equivalents here.

Examples

Publish `reboot` ; the device replies with the [code 1](#) accept under [Response Payload](#), then runs the [reboot cycle](#). If an inventory is active, the device refuses the reset with [code 5](#) :

Response — refused during inventory (code 5)

```

1  {
2      "command" : "reboot" ,
3      "requestId" : "123abcd" ,
4      "apiVersion" : "V1.1" ,
5      "response" : {
6          "code" : 5 ,
7          "description" : "Can't reboot device, inventory in progress"
8      }
9  }
```

Notes and Caveats

⚠ SESSION-SEVERING — SEND ONCE, NEVER AUTO-RETRY

The warm reset drops the control session; the device auto-reconnects to the previously-connected server, but recovery time **varies** (~30 s–2 min observed). Because a reboot cannot be un-done and interrupts service, issue it deliberately (ideally after operator confirmation) and **never auto-retry** it — the reboot may already have taken effect even if the accept was lost.

- **Endpoint config persists; operating mode does not.** Across the reset, saved and active MQTT endpoints are restored and re-activated, but the **operating mode reverts to default** (any tuned profile or armed `accessOperations` is cleared). Re-apply the operating mode after a reboot if you were relying on a non-default configuration — see `set_operating_mode`.

Sequence a reboot as: (optionally) stop any inventory with `control_operation` → send `reboot` once → wait out the reconnect (poll, do not hard-code the delay) → confirm with `get_version` / `get_status`. For the concept-level view, see [Device Lifecycle](#) and [Broker Connection](#).

Related:  [Device Lifecycle](#) ·  `set_os` ·  `get_version` ·  `control_operation` ·  [Response Codes](#)